Ministry of Higher Education and Research
Higher School of Computer Science 08 May 1945 - Sidi Bel Abbes

Second Year Second Cycle - Artificial Intelligence and Data Science

---

Project Report

SYMPTOM CHECKER WITH MEDICAL TEXT CLASSIFICATION

Students:

- Souad BOUKHAROUBA
- Sanaa BENSAID
- Lamia ABDELMALEK

Supervisor:

Dr. Rabab Bousmaha

Release date: May 6, 2025

Table of contents

1 Introduction **3**

2 Dataset **3**

3 Data Preprocessing **4**

4 Data Splitting **4**

5 Models Used **4**

6 Training **5**

7 Evaluation **5**

8 Prediction Results **7**
8.1 LSTM Model Results 7
8.2 GRU Model Results 7
8.3 Advanced Baseline Model Results 7

9 Deployment **8**

10 Conclusion **9**

List of Figures

1 Preview of the dataset 3
2 Class distribution in the dataset 4
3 Number of classes in the dataset 4
4 LSTM: The validation accuracy quickly surpasses training accuracy, indicating good generalization. Loss steadily decreases. 6
5 GRU: Similar behavior to LSTM but with slightly better final accuracy. Training and validation curves show consistent learning. 6
6 Baseline Feedforward: Shows faster convergence and higher validation accuracy, indicating that even a simpler model can generalize well. 6
7 Advanced Dual-Input Model: Achieves the best performance with closely matched training and validation curves for both accuracy and loss, indicating excellent generalization. 7
8 Streamlit app interface: User input for symptoms. 8
9 Streamlit app interface: Model predictions and results. 9

1 Introduction

The main goal of this project is to apply Natural Language Processing (NLP) techniques to classify medical symptoms into their respective categories. This involves several key steps: text preprocessing, feature extraction, the use of sequence models, and deep learning architectures. By leveraging these methods, we aim to develop an accurate and robust system for medical symptom classification, which can be a valuable component in digital healthcare applications.

2 Dataset

Name: `mohammad2928git/complete_medical_symptom_dataset`

This dataset includes medical symptom descriptions and their corresponding disease labels.

- **Number of rows:** Approximately 1.3 million
- **Columns:**
  - `symptoms` – textual description of the symptoms
  - `disease` – the medical category or condition associated

symptoms sequence · *lengths* 1 430	**lebel_text** sequence · *lengths* 1 34
["migraines headaches", " can't sleep", " whole body shaking shivering", " feel dizzy some…	["drug reaction"]
["asthma wheezing breathing problems", " fevers", " headaches", " feel tired time", ""]	["allergy"]
["signs symptoms primary ovarian insufficiency similar menopause estrogen deficiency", " i…	["premature ovarian failure"]
["cough", "high fever", "breathlessness", "family history", "mucoid sputum"]	["bronchial asthma"]
["chills", "vomiting", "high fever", "sweating", "headache", "nausea", "diarrhoea", "muscl…	["malaria"]
["posterior cortical atrophy symptoms vary people", " symptoms can vary time", " tend grad…	["posterior cortical atrophy"]
["i've back pain", " cough", " numbness arms legs", " neck hurts too", " i've feeling dizz…	["cervical spondylosis"]

Figure 1: Preview of the dataset

3 Data Preprocessing

We applied several NLP preprocessing techniques to facilitate working with the dataset. Since the dataset was already fairly clean, we applied only the following preprocessing steps:

- Removed null values.
- Converted all text to lowercase.
- Removed special characters and extra spaces.
- Applied lemmatization to normalize words.

The dataset was imbalanced, with a disproportionate number of samples for some classes. This could affect model training, so we visualized the class distribution and the number of classes in the dataset.

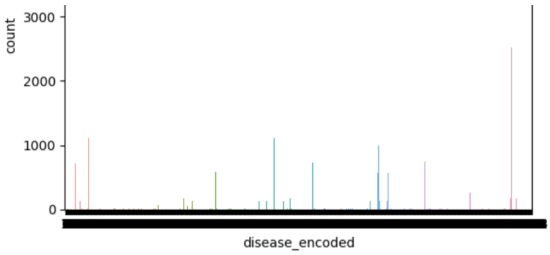


Figure 2: Class distribution in the dataset

```
Total unique symptoms: 18434

Top 10 symptoms:
feel pain                                                          127724
characterize pain:                                                  65831
pain radiate location                                               54110
traveled country last 4 weeks                                       42557
intense pain                                                        31822
precisely pain located                                              31822
fast pain                                                           31822
pain reason consulting                                              31386
affected region located                                             24088
experiencing shortness breath difficulty breathing significant way  16351
dtype: int64
```

Figure 3: Number of classes in the dataset

To handle class imbalance, we applied class weights during model training, which helped ensure the model gave more attention to underrepresented classes.

4 Data Splitting

The dataset was split into:

- 70% training data
- 30% validation data

5 Models Used

We experimented with a variety of deep learning models for the classification task, including sequence models and custom architectures:

1. LSTM (Long Short-Term Memory)

LSTM is a type of recurrent neural network (RNN) capable of learning long-term dependencies in sequential data. It uses memory cells and gating mechanisms (input, forget, and output gates) to retain important information over long input sequences.

2. GRU (Gated Recurrent Unit)

GRU is a simplified version of LSTM that combines the forget and input gates into a single update gate. It is computationally more efficient than LSTM while maintaining comparable performance on many sequence tasks.

3. Baseline Feedforward Model

A simple fully connected feedforward neural network was implemented to establish baseline performance. It consists of several dense layers with dropout regularization, trained using bag-of-words features.

4. Advanced Dual-Input Model (BOW + LSTM)

We also implemented a hybrid model that combines:

- A **Bag-of-Words (BOW)** branch for capturing general term presence.
- A **Bidirectional LSTM** branch for learning sequential dependencies in the text.

These two branches are concatenated and passed through fully connected layers before final classification. This approach leverages both frequency-based and sequence-based text representations.

6 Training

We trained each model using appropriate input formats and training strategies:

- Used tokenized sequences or embedding representations depending on the model architecture.
- Applied class weights to address the class imbalance issue during training.
- Monitored validation metrics to prevent overfitting and guide training.
- For **LSTM** and **GRU** models, training was performed for 20 epochs.
- For the **Baseline Feedforward** and **Advanced Dual-Input** models, training was done for 15 epochs.
- **Accuracy** was used as the primary evaluation metric during training.
- The **categorical_crossentropy** loss function was used for all models since the task is multi-class classification.

7 Evaluation

Each model was evaluated using classification performance metrics. Below is a comparison of their final validation accuracy and loss:

Model	Validation Accuracy	Validation Loss
LSTM	72.64%	1.41
GRU	73.5%	1.30
Baseline Feedforward	80%	0.60
Advanced Dual-Input (BoW + LSTM)	**89.43%**	**0.31**

Table 1: Comparison of model performance on the validation set

Below are the accuracy and loss plots for the models during training:

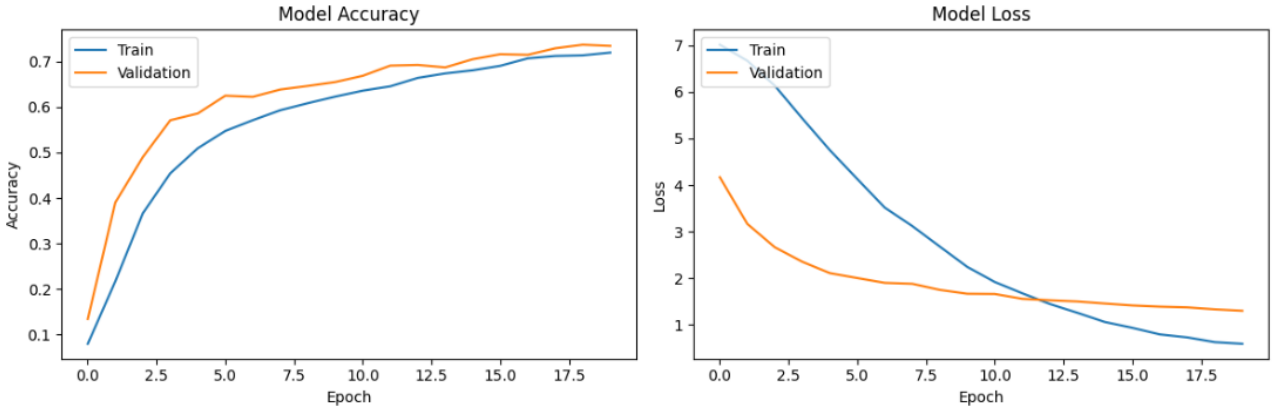


Figure 4: LSTM: The validation accuracy quickly surpasses training accuracy, indicating good generalization. Loss steadily decreases.

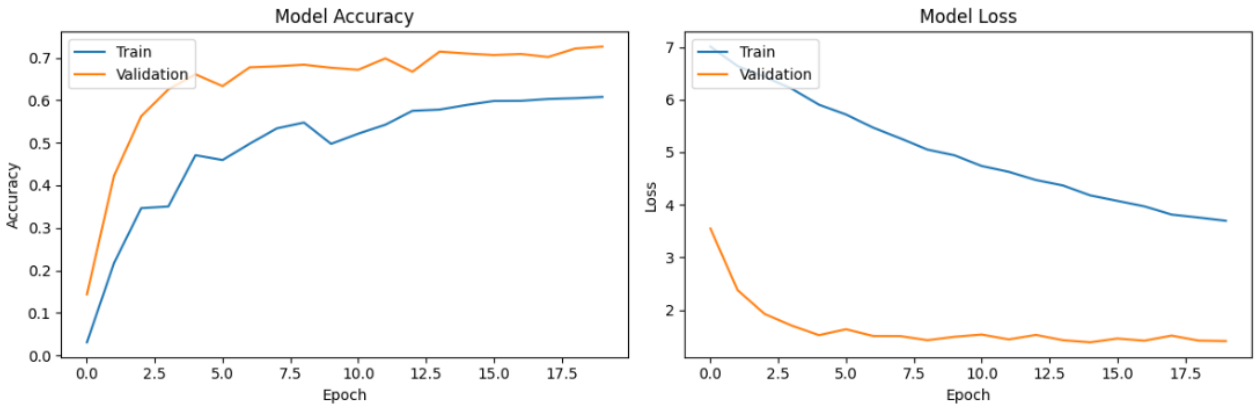


Figure 5: GRU: Similar behavior to LSTM but with slightly better final accuracy. Training and validation curves show consistent learning.

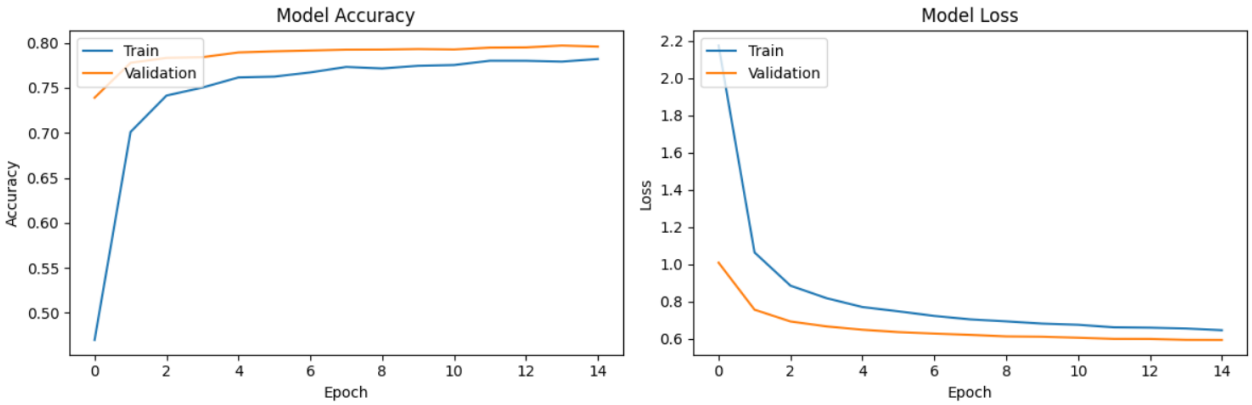


Figure 6: Baseline Feedforward: Shows faster convergence and higher validation accuracy, indicating that even a simpler model can generalize well.

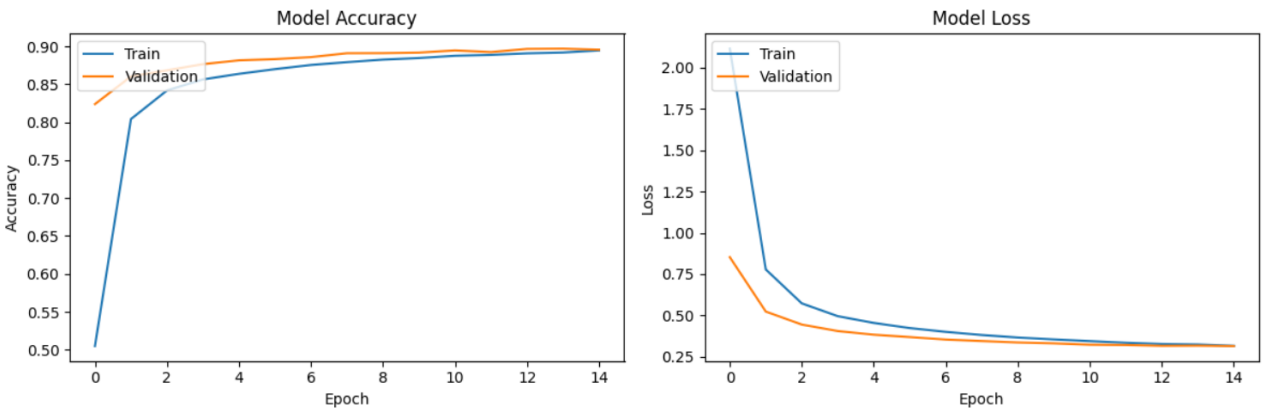


Figure 7: Advanced Dual-Input Model: Achieves the best performance with closely matched training and validation curves for both accuracy and loss, indicating excellent generalization.

The Advanced Dual-Input model (BoW + LSTM) achieved the best overall results. Its validation accuracy closely matched the training accuracy, and the validation loss stayed aligned with the training loss throughout, indicating strong generalization and low overfitting.

8 Prediction Results

In this section, we display the predictions made by the models (LSTM, GRU, and Advanced Baseline) along with their respective confidence scores.

8.1 LSTM Model Results

Symptom Input: I have fatigue, yellowish skin, nausea, yellowing of eyes, family history.

Disease Predictions:

- Disease: **hepatitis C**, Probability: 0.5792
- Disease: **alcoholic hepatitis**, Probability: 0.2722
- Disease: **chronic cholestasis**, Probability: 0.0748

8.2 GRU Model Results

Symptom Input: The individual is experiencing various allergy symptoms, which can affect different body systems. Common symptoms include sneezing, itching, runny nose, watery or swollen eyes (conjunctivitis), and throat swelling. Allergic reactions may include food allergies, insect sting allergies, drug allergies, and skin conditions like eczema. Severe allergic reactions can lead to anaphylaxis, with symptoms like loss of consciousness, low blood pressure, severe shortness of breath, skin rash, and rapid pulse.

Disease Predictions:

- Disease: **allergies**, Probability: 0.9758
- Disease: **bile reflux**, Probability: 0.0012
- Disease: **concussion**, Probability: 0.0010

8.3 Advanced Baseline Model Results

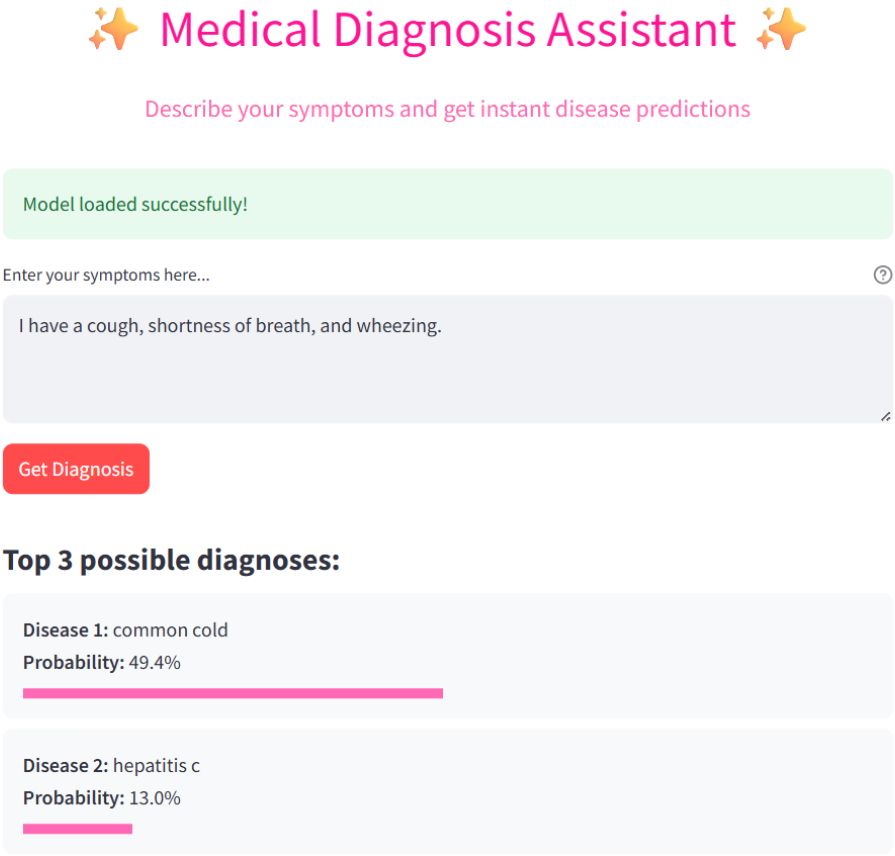
Symptom Input: I have a cough, shortness of breath, and wheezing.

Disease Predictions:

- Disease: **allergy**, Probability: 0.6291
- Disease: **common cold**, Probability: 0.1143
- Disease: **pneumonia**, Probability: 0.0927

9 Deployment

To deploy the model, we used Streamlit, a Python-based framework for creating interactive web applications.We created a simple chat interface where users can input their symptoms, and the model will return predicted diseases with corresponding probabilities. This application provides an intuitive interface that allows users to easily interact with the model and receive medical predictions based on the symptoms they provide.
Below are screenshots of the deployed application:



path

Figure 8: Streamlit app interface: User input for symptoms.

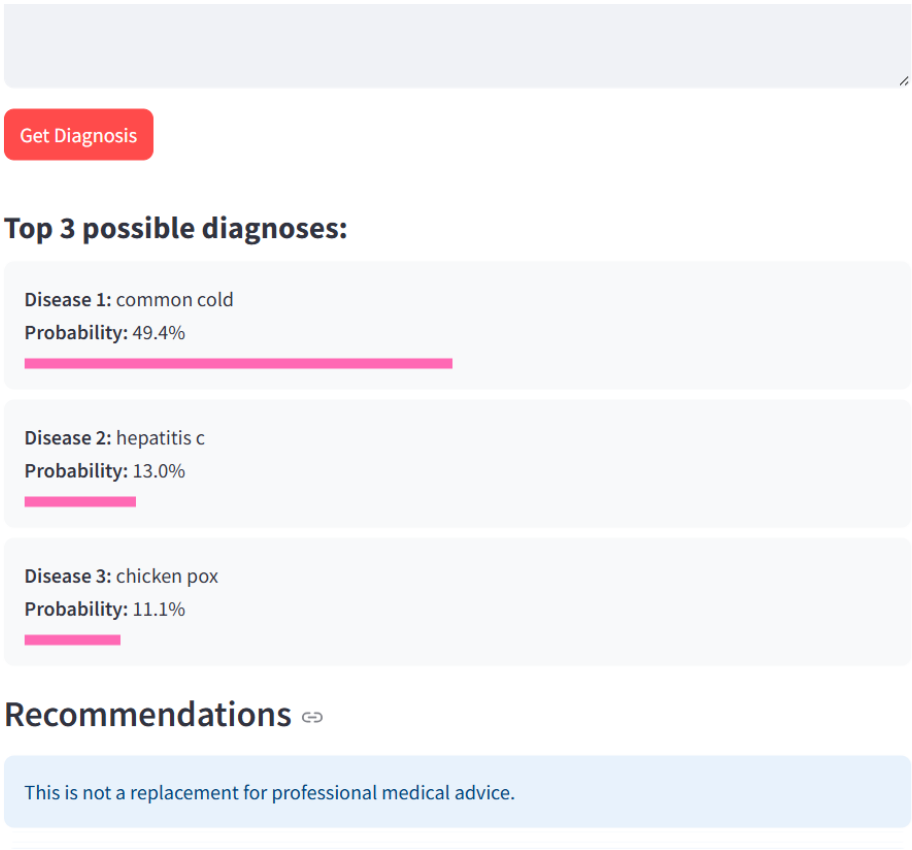


Figure 9: Streamlit app interface: Model predictions and results.

10 Conclusion

This project helped us understand Natural Language Processing (NLP) techniques by applying them to the classification of medical symptoms. By implementing and evaluating different deep learning models, we gained practical insights into how text data can be used for disease prediction. Future work could involve expanding the dataset, improving model interpretability, and incorporating additional features such as patient history or symptom duration.